

CivicRecords AI

Open-source, locally-hosted AI that helps American cities respond to open records requests.

> **Release recovery notice (2026-05-15).** CivicRecords AI v1.6.1 is the ingestion worker recovery patch on top of the v1.6.0 Docker secret-file extraction release and the v1.5.0 CivicCore v1.0.1 recovery alignment. The older `v1.4.10` tag remains available as historical source only and must not be promoted as an attested baseline.

CivicRecords AI runs entirely on a single machine inside your city's network — no cloud subscriptions, no vendor lock-in, no resident data leaving the building. It ingests your city's documents, makes them searchable with AI-powered natural language queries, detects potential exemptions, and manages the full request lifecycle from intake to response.

Why This Exists

Every city in America processes open records requests (FOIA, CORA, and state equivalents). Staff manually search file shares, email archives, and databases — then review every document for exemptions before release. It's slow, error-prone, and a growing burden as request volumes increase.

No open-source tool exists for the **responder side** of open records at the municipal level. CivicRecords AI fills that gap.

Key Features

- **AI-Powered Search** — Natural language hybrid search (semantic + keyword) across all ingested city documents with source attribution, normalized relevance scores, and optional AI-generated summaries
- **Document Ingestion** — Automatic parsing of PDF, DOCX, XLSX, CSV, email, HTML, and text files. Scanned documents processed via multimodal AI (Gemma 4) with Tesseract OCR fallback
- **Exemption Detection** — Tier 1 PII detection (SSN, credit card with Luhn validation, phone, email, bank accounts, state-specific driver's licenses) plus per-state statutory keyword matching. Optional LLM secondary review. All flags require human confirmation
- **Request Management** — Full lifecycle tracking with 10 statuses: received, clarification needed, assigned, searching, in review, ready for release, drafted, approved, fulfilled, closed.

Timeline, messaging, fee tracking, and response letter generation

- **Guided Onboarding** — Two modes operators can switch between: a 3-phase form wizard (City Profile → Systems → Gap Map), and a single-phase LLM-powered adaptive interview that persists each answer (including `has_dedicated_it`) to the CityProfile singleton and drives the `onboarding_status` lifecycle (`not_started` → `in_progress` → `complete`). Skip advances the walk truthfully. Both modes surface coverage gaps across 12 municipal domains.
- **Municipal Systems Catalog** — Curated knowledge base of 25+ municipal software vendors across 12 functional domains (finance, public safety, permitting, HR, etc.) with discovery hints and connector templates
- **Universal Connector Framework** — Standardized protocol (`authenticate/discover/fetch/health_check`) for connecting to city data sources. Ships with four implemented connector types: `file_system` (local/mounted directories), `manual_drop` (watched drop folders), `rest_api` (generic REST API — API key / Bearer / OAuth2 client-credentials / Basic auth; JSON/XML/CSV; page/offset/cursor pagination), and `odbc` (SQL databases via pyodbc, row-as-document with SQL-injection guards). IMAP email, SMB/NFS, and SharePoint connectors on roadmap
- **Scheduled Sync & Idempotent Ingestion** — Per-source cron scheduling (5-field expressions via croniter, evaluated in UTC with local-time disclosure, rolling 7-day min-interval validation, 5-minute floor) with `schedule_enabled` pause toggle. Idempotent pipeline: binary sources dedup by content hash, structured REST/ODBC sources dedup by stable source-path with canonical JSON serialization. Concurrent-update races prevented via `SELECT FOR UPDATE` + partial UNIQUE indexes; content updates atomically replace chunks and embeddings in the same transaction
- **Sync Failure Tracking & Circuit Breaker** — Per-record failure tracking with two-layer retry (task-level exponential backoff + record-level per-tick retry with $N=100/T=90s$ cap). Automatic circuit breaker after 5 consecutive full-run failures (`sync_paused`) with admin-feedback unpause grace period. `health_status` (healthy/degraded/circuit_open) computed live from failure counts. Admin UI: colored health badge, failed records panel with bulk retry/dismiss, Sync Now button with real-time polling progress
- **Operational Analytics** — Real-time metrics: average response time, deadline compliance rate, overdue requests, status breakdown
- **Notification Service** — Template-based notification system with SMTP email delivery via Celery beat (60s interval). Configure `SMTP_HOST`, `SMTP_PORT`, `SMTP_USERNAME`, `SMTP_PASSWORD` in `.env` to enable. Notification dispatch into status transitions pending
- **Compliance by Design** — Hash-chained audit logs, human-in-the-loop enforcement, AI content labeling, data sovereignty verification. Designed for Colorado CAIA and 50-state regulatory compliance. CJIS compliance gate for public safety connectors
- **Civic Design System** — Professional UI built with shadcn/ui and civic blue design tokens. Responsive shell: fixed 240px sidebar at $\geq 768px$, hamburger-driven slide-in drawer below

that (with focus trap, ESC close, overlay dim, auto-close on route change). WCAG 2.2 AA targeted (44px touch targets, skip-to-content link, icon+color status badges, programmatic label→input associations on admin forms, `role="alert"` validation errors with actionable copy — full third-party accessibility audit still pending)

- **Federation-Ready** — REST API with service accounts enables future cross-jurisdiction record discovery between CivicRecords AI instances

Quick Start

Requirements

CivicSuite is a **server product** — one server per city. Clerks reach the server through any modern browser; there is no clerk-side installation.

CivicSuite Server (one per city)

Runs the full Docker Compose stack: postgres, redis, ollama, api, celery worker, celery beat, and frontend.

- **CPU:** 8+ cores recommended.
- **RAM:** the installer gates on `MIN_RAM_GB` from `scripts/detect_hardware.sh` (32 GB at the time of writing). That value is flagged "needs measurement — arbitrary value pending peak-RAM profile" in its provenance comment; see the comment block at the top of `scripts/detect_hardware.sh` for the rationale and the intended revision path.
- **Disk:** 50 GB free.
- **Host OS:** Linux (Ubuntu/Debian) via `install.sh`, Windows 10/11 via `install.ps1`, or macOS via `install.sh` (script path; not lifecycle-certified). Docker Desktop / Docker Engine must already be installed and running.
- **Network:** No outbound internet connection required after initial setup.
- **Hardware cost (secondary market):** Refurbished Dell OptiPlex small-form-factor units with 16–32 GB RAM are commonly available and are a plausible hardware-cost target for a city-scale deployment. Confirm against `MIN_RAM_GB` (currently 32 GB) before purchasing on the low end of that range.

Clerk Workstation (one per clerk)

Any computer with a modern web browser. Clerks open the server's URL —

`http://<server>:8080` — sign in, and use the application. **Zero client-side install** is required: no Docker, no helper script, no Python runtime, no model weights downloaded to the clerk machine. The server hosts everything.

- **Browser:** Chromium-based (Chrome, Edge), Firefox, or Safari — current version.

- **OS:** any (Windows, macOS, Linux, ChromeOS — anything that runs a modern browser).
- **Network:** LAN access to the server on ports 8080 (web UI) and 8000 (API).
- **Install:** none.

Install

> **Install paths.** Two options ship today: > > 1. **Windows double-click installer (T5E, UNSIGNED).** A signed build is a future release — this one is not. The unsigned installer is published on every release tag at `releases/download/<tag>/CivicRecordsAI-<version>-Setup.exe` along with a SHA-256 checksum for independent verification. On first run Windows SmartScreen shows **"Windows protected your PC — Unknown publisher."** This is expected. Click **More info** → **Run anyway** to proceed. See [installer/windows/README.md](https://github.com/CivicSuite/civicrecords-ai/blob/main/installer/windows/README.md) for the full SmartScreen walkthrough and checksum-verify steps. The installer bundles the repo snapshot, runs a prereq check (Docker Desktop, WSL 2 + Virtual Machine Platform, 32 GB RAM floor, optional host Ollama), then runs `install.ps1` (via `installer\windows\launch-install.ps1`). `install.ps1` **auto-pulls nomic-embed-text** and **auto-pulls the Gemma 4 tag you select in the picker** (default `gemma4:e4b`) — expect several minutes on first run — and seeds the T5B baseline datasets. > > 2. **Script-based install (Linux / macOS — not lifecycle-certified — and Windows if you prefer CLI).** Windows-only currently; macOS support pending lifecycle certification. The scripts below configure and start the Docker Compose stack on macOS and Linux as a non-certified path, and on Windows as a CLI alternative. They do **not** install Docker, WSL, or any other system prerequisites — those must already be present. `install.ps1` / `install.sh` both ship the 4-model Gemma 4 picker, auto-pull the selected LLM plus `nomic-embed-text`, and auto-seed the baseline datasets on first boot. > > **Two shortcuts, two flows.** The Windows installer creates **separate** Start Menu entries for the two operations — don't confuse them: > > - **Start CivicRecords AI** → daily start. Runs `docker compose up -d` and opens `http://localhost:8080/`. Does **not** run the prereq check, does **not** invoke `install.ps1`, does **not** pull any model, does **not** re-seed data. The Desktop shortcut (if you opted in) mirrors this daily-start behavior. > > - **Install or Repair CivicRecords AI** → full bootstrap/repair. Runs the prereq check, then `install.ps1` (which may show the picker and pull models). Use this for first-run setup (the installer fires it automatically for you the first time), when you want to switch LLMs, or to repair a broken stack. > > **Docker Desktop and WSL 2** must be installed and running before either path; the installer detects their absence and prints concrete remediation, but does not install them for you.

Windows: `powershell git clone https://github.com/CivicSuite/civicrecords-ai.git`
`cd civicrecords-ai .\install.ps1``

```
macOS / Linux (script path; not lifecycle-certified – see "Supported Platforms" below): `bash git clone https://github.com/CivicSuite/civicrecords-ai.git cd civicrecords-ai bash install.sh `
```

First Use

1. Open **http://localhost:8080** in your browser 2. Sign in with the admin credentials you configured in `.env` 3. Go to **Sources** → **Add Source** → enter a directory path to your documents 4. Click **Ingest Now** – documents are parsed, chunked, and indexed automatically 5. Go to **Search** – type a natural language query and get cited results

Phase 1 migration layer

CivicRecords AI backend installs `civiccCore` (the shared CivicSuite schema + migration runtime) as a dependency. During the release-hardening window before PyPI publication, `backend/pyproject.toml` points at a versioned GitHub release wheel rather than a Git SHA. This keeps the dependency reproducible without requiring `git` inside the backend image. The current release line targets the published `CiviccCore v1.0.1` wheel so `records-ai` can consume shared search, onboarding, connector security, startup config validation, schedule validation, datasource source-list status projection, ingest contracts, persisted audit-log hash/verification primitives, live connector retry/circuit-breaker state transitions, vendor-delta request planning, and reusable mock-city contract suites from `CiviccCore` instead of carrying local copies. The shipped `civiccCore` dependency surface already covers the LLM provider abstraction, prompt-template engine, model registry, audit primitives, provenance contracts, city-profile schema, connector manifests, export-bundle helpers, onboarding interview field/order lifecycle contract, connector security/config substrate, storage-neutral persisted audit-log verification, reusable live-sync operator-state helpers, source-list status projection helpers, vendor delta URL planning, reusable startup config validation and cron schedule helpers, and reusable mock municipal environment contracts for connector, IdP, and backup-retention rehearsals.

Migrations run in two layers: `civiccore` first (creates/updates the 16 shared tables), then this repo's Alembic chain on top. See [ADR-0003](#) for the full gate contract.

Release provenance

CivicRecords AI now wires release preflight to CivicCore's canonical `civiccore.release_provenance` helper. This matters because GitHub release pages can show the target commit as "Verified" even when the release tag is lightweight or unsigned. Treat that badge as a commit signal only; the actual trust artifact for post-baseline releases is the Sigstore-signed `release-attestation.json` plus bundle documented in [docs/ops/release-signing.md](#).

The older public `v1.4.10` release is a historical pre-gate, provisional artifact because its tag predates the attestation model and its public release assets do not include `release-attestation.json` or `release-attestation.json.bundle`. C0-4 records the decision in [docs/ops/tier1-retrofit-ledger.md](#): do not republish, mirror, or promote `v1.4.10` as an attested provenance baseline.

Architecture

Deployment stack

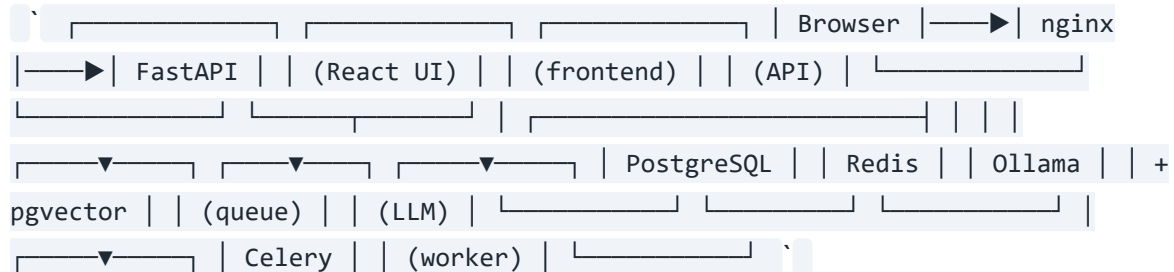
 Deployment stack – entire system runs inside Docker Compose on the city's network. No cloud, no outbound by default.

LLM call flow

 LLM call flow: records-ai application code routes through `civiccore.llm` (context assembly, template resolution, model registry, provider factory) to a local Ollama provider, with optional cloud providers.

Sovereignty / data boundary

 Sovereignty boundary: all runtime components (FastAPI, Celery, Postgres+pgvector, Ollama, local volumes) live inside the city's on-prem network. No outbound by default; cloud is opt-in only.



7 Docker services: PostgreSQL 17 + pgvector, Redis 7.2, Ollama, FastAPI, Celery worker, Celery beat, nginx frontend.

Tech stack: Python 3.12, FastAPI, SQLAlchemy 2.0, React 18, shadcn/ui, Tailwind CSS, Alembic, Celery, pgvector, nomic-embed-text, Gemma 4 (recommended).

Configuration

Most configuration is via environment variables in `.env`. Secret material for JWT signing and the initial admin password is file-backed in v1.6.0 and is not placed in the container environment:

Variable	Description	Default	
DATABASE_URL	PostgreSQL connection string	postgresql+asyncpg://civicrecords:civicrecords@postgres:5432/civicrecords	
FIRST_ADMIN_EMAIL	Initial admin account email	admin@example.gov	
CIVICRECORDS_SECRET_DIR	Host directory containing file-backed secrets for Docker Compose	./data/secrets	
OLLAMA_BASE_URL	Ollama API endpoint	http://ollama:11434	
REDIS_URL	Redis connection string	redis://redis:6379/0	
AUDIT_RETENTION_DAYS	Audit log retention period	1095 (3 years)	
CONNECTOR_HOST_ALLOWLIST	Comma-separated hostnames/IPs exempt from SSRF block (on-prem use) (empty – RFC1918, loopback, and cloud IMDS ranges blocked by default)		

Operations / Secrets

As of v1.6.0, `JWT_SECRET` and `FIRST_ADMIN_PASSWORD` are no longer runtime container environment variables. The install scripts generate the same strong random values as earlier releases, write them to `./data/secrets/jwt_secret` and `./data/secrets/first_admin_password`, and configure Docker Compose to mount those files at `/run/secrets/...`. The container environment intentionally does not include `JWT_SECRET`, `FIRST_ADMIN_PASSWORD`, or matching `_FILE` pointer names. A release rehearsal must prove:

```
` bash docker exec <records-api-container> env | grep -E  
"JWT_SECRET|FIRST_ADMIN_PASSWORD" `
```

returns zero lines.

Upgrade note from v1.5.x: existing installs that still have `JWT_SECRET=...` or `FIRST_ADMIN_PASSWORD=...` in `.env` must re-run `install.sh` or `install.ps1` so the secret files are created and `.env` keeps only non-secret operator settings such as `CIVICRECORDS_SECRET_DIR`. Do not set `JWT_SECRET`, `FIRST_ADMIN_PASSWORD`, or matching `_FILE` pointer env vars in Docker deployments; those names are recoverable with `docker exec env`.

Supported Platforms

- Windows 10/11 (Docker Desktop)
- macOS 13+ (Docker Desktop) – Windows-only currently; macOS support pending lifecycle certification (script-path install only)
- Ubuntu 22.04+ / Debian 12+ (Docker Engine)

All platforms use identical Docker containers – the application runs in Linux containers regardless of host OS.

Data Sovereignty

CivicRecords AI is designed for environments where resident data must never leave the network:

- Runs entirely on local hardware – no cloud dependencies
- No telemetry, analytics, or crash reporting
- All LLM inference runs locally via Ollama
- Live verification script confirms the running Compose stack is local-only: start the stack with `docker compose up -d`, then run `bash scripts/verify-sovereignty.sh`

License

Apache License 2.0 – see [LICENSE](#).

All dependencies use permissive (MIT, Apache 2.0, BSD) or weak-copyleft (LGPL, MPL) licenses. No AGPL, SSPL, or BSL dependencies.

Documentation

Complete System Manual (unified staff + IT admin guide with architecture diagrams):

- [Download PDF](#) | [Download Word](#) | [View Online](#)

Individual References:

- [Staff User Manual](#) – For city clerks and records officers (non-technical)
- [IT Administrator Manual](#) – Installation, configuration, security, backup, monitoring
- [Canonical Spec](#) – Unified Design Specification (single source of truth)
- [System Architecture Diagram](#) – Interactive component and data flow diagrams
- [Phase Decomposition](#) – Project phases and build sequence

Contributing

See [CONTRIBUTING.md](#) for development setup, coding standards, and how to submit changes.

Testing Prerequisites

v1.6.1 (May 15, 2026) – Celery ingestion worker event-loop recovery patch. Worker tasks now create and dispose their async SQLAlchemy engine inside each task coroutine instead of reusing a module-global engine across Celery prefork task event loops. This removes the `RuntimeError: Event loop is closed` / `Future attached to a different loop` failure mode that could leave later ingests stuck in `pending` after the first successful worker task. Release-prep evidence collected 640 backend tests and 36 frontend Vitest tests.

v1.5.0 (May 10, 2026) – CivicCore recovery alignment release. Records-AI now consumes civiccore v1.0.1, matching the active CivicSuite platform baseline and closing ENG-002. The imported CivicCore symbols remained compatible, so this release changes the dependency baseline and release evidence without changing API URL paths, roles, permissions, or records-side database migrations.

v1.6.0 (May 11, 2026) – Docker secret-file extraction release. `JWT_SECRET` and `FIRST_ADMIN_PASSWORD` move out of container env vars and into Docker-mounted secret files. Existing v1.5.x installs must re-run `install.sh` or `install.ps1` so `./data/secrets/*` is created and `.env` keeps only non-secret operator settings; `_FILE` pointer env names are intentionally absent from the container env.

v1.4.10 (May 3, 2026) – Documentation-only release alignment patch. The v1.4.9 CivicCore source-status projection code is unchanged; this tag publishes the audit-corrected 631-backend-test evidence in the release source snapshot and installer/download docs.

v1.4.9 (May 3, 2026) – CivicCore source-status projection consumer patch. Records-AI now consumes civiccore v0.22.0 for datasource list health, active-failure count, pause, and next-run projection, preserving the existing `/datasources/` response shape while moving reusable operator-state semantics into CivicCore. API URL paths, roles, permissions, and records-side database migrations are unchanged from v1.4.8.

v1.4.8 (May 2, 2026) – CivicCore schedule validation consumer patch. Records-AI now consumes civiccore v0.21.0 for cron schedule validation and next-run computation, preserving the local `app.ingestion.cron_utils` import surface as a

compatibility wrapper. API URL paths, roles, permissions, and records-side database migrations are unchanged from v1.4.7.

v1.4.7 (May 2, 2026) – CivicCore startup config validation consumer patch. Records-AI now consumes civiccore v0.20.0 for JWT secret, Fernet encryption key, first-admin password, and CSV allowlist parsing validation, on top of the v1.4.6 vendor-delta/mock-city extraction. API URL paths, roles, permissions, and records-side database migrations are unchanged from v1.4.6.

v1.4.6 (May 2, 2026) – CivicCore mock-city/delta consumer patch. Records-AI consumes civiccore v0.19.0 for vendor-delta request planning and reusable mock-city contract suites, on top of the v1.4.5 live connector retry and circuit-breaker extraction. API URL paths, roles, permissions, and records-side database migrations are unchanged from v1.4.5.

v1.4.5 (May 2, 2026) – CivicCore sync-consumer patch. Records-AI consumes civiccore v0.18.1 for live connector retry and circuit-breaker primitives, preserving the existing Records retry semantics and database-backed sync failure workflow while removing reusable local sync-state logic. API URL paths, roles, permissions, and records-side database migrations are unchanged from v1.4.4.

v1.4.4 (May 1, 2026) – CivicCore audit-consumer patch. Records-AI now consumes civiccore v0.17.0 for persisted audit-log hashing and verification, in addition to shared search, onboarding, connector-security, and ingest contracts. API URL paths, roles, permissions, and records-side database migrations are unchanged from v1.4.3.

v1.4.3 (April 29, 2026) – Shared connector-security extraction patch. Records-AI now consumes civiccore v0.13.0 for connector host validation and encrypted-config primitives instead of carrying private copies of those helpers. API URL paths, roles, permissions, and records-side database migrations are unchanged from v1.4.1.

v1.4.0 (April 25, 2026) – Phase 2 LLM integration. civiccore advanced from v0.1.0 to v0.2.0; LLM provider abstraction, prompt-template engine + 3-step override resolver, and model registry now sourced from civiccore.llm. Migration 020_phase2_consumer_app_backfill runs after upgrade. Records-AI now consumes civiccore v0.2.0 as a versioned dependency.

v1.3.0 – 2026-04-25 release. Phase 1 CivicCore extraction landed: civiccore v0.1.0 is now consumed as a release-wheel dependency. Two-layer migration order –

civiccore migrations run first via subprocess, then records-side. No API or UI changes (infrastructure only). See [CHANGELOG](#) and the v1.3.0 release notes for operator upgrade guidance.

v1.2.0 – 2026-04-23 release. Tier 5 (installer + onboarding + seeding + model picker + portal mode) and Tier 6 (at-rest encryption, ENG-001 closed) ship together. CI green on `d556904` (run 24853147133). Backend 617/617 pytest, frontend 36/36 vitest, unsigned Windows installer produced on tag push.

v1.1.0 – Phase 2 release with department access controls, 50-state exemption rules, and compliance templates.

New in v1.1.0:

- Department-level access controls – staff scoped to own department, admins see all
- Department CRUD API with audit logging
- 50-state + DC exemption rule coverage (175 jurisdiction-scoped rules across 51 jurisdictions, plus 5 universal PII rules available in the seed source – 180 total)
- 5 compliance template documents (AI Use Disclosure, Response Letter Disclosure, CAIA Impact Assessment, AI Governance Policy, Data Residency Attestation)
- Template render endpoint with city profile variable substitution
- Exemption auditability dashboard with acceptance/rejection rates and CSV/JSON export (time-period filtering not yet implemented)
- Model registry CRUD endpoints (spec 6.7 compliance metadata)

Carried from v1.0.x:

- 13 staff workbench pages + Login with shadcn/ui design system
- 29 database tables, ~30 API endpoints
- 631 backend + 36 frontend automated tests passing (CI-verified on PR #61, run 25272823105)
- Guided onboarding, systems catalog, connector framework
- Request timeline, messaging, fee tracking, response letter generation
- Operational analytics and notification service
- AMD GPU/NPU hardware auto-detection (ROCm on Linux, DirectML on Windows)
- Login rate limiting, audit log archival, admin-only user creation
- Tested on Windows 11 (Docker Desktop) and Ubuntu 22.04 (Docker Engine)

In v1.2.0 – Tier 5 (all five slices shipped 2026-04-22 → 2026-04-23, tagged in this release):

- T5C – 4-model Gemma 4 installer picker, 2026-04-22 (7721cf0):** Picker now shows exactly four supported Gemma 4 tags: `gemma4:e2b` , `gemma4:e4b` (default), `gemma4:26b` , `gemma4:31b` – with per-model disk footprint, min/recommended RAM advisories, and a `supportable_against_target` boolean against the locked Windows 11 / 32 GB baseline. The fake tags `gemma4:12b` and `gemma4:27b` that contaminated `install.sh` , `install.ps1` , `scripts/detect_hardware.*` , and `backend/app/config.py` have been purged repo-wide. Host RAM is re-verified empirically at install time regardless of picker selection.
- T5A – Onboarding persistence, 2026-04-22 (1782573):** The single-phase LLM-powered adaptive interview now actually persists each answer onto the `CityProfile` singleton (creating the row on the first answer), normalizes yes/no → bool for `has_dedicated_it` , and transitions `onboarding_status` (`not_started` → `in_progress` → `complete`). Skip advances the walk truthfully (previously it dropped answers silently). Coverage: 4 new persistence tests + 2 skip-truth regression tests; migration `018_city_profile_state_nullable.py` relaxes a constraint the persistence path needed.
- T5B – First-boot baseline seeding, 2026-04-22 (61449c5):** `app/main.lifespan` now auto-seeds three baseline datasets on first boot: **175 state-scoped exemption rules across 51 jurisdictions** (from `STATE_RULES_REGISTRY`), **5 compliance templates**, and **12 notification templates**. Idempotent via `skip-if-exists` – admin customizations survive re-seed. Every run emits a start line, per-dataset `created / skipped` counts, and a completion summary. Universal PII regex rules are present in the seed source (5 additional rules, 180 total) but are **not** seeded by first-boot because they lack a two-letter `state_code` ; those remain deferred pending schema relaxation. See `backend/app/seed/first_boot.py` and §8.7 of the spec.
- T5D – Install-time portal switch (private vs. public), 2026-04-23 (a57a897):** New `PORTAL_MODE` environment variable (`private` | `public` , default `private`) locked at install time, changeable post-install by editing `.env` and restarting the stack. The installer (`install.ps1` / `install.sh`) prompts for the choice interactively; non-interactive installs can pre-set `CIVICRECORDS_PORTAL_MODE` . Case and whitespace are normalized by a `field_validator` on the config model. **Private mode (default)** is staff-only – the login screen is the only externally reachable page, `/auth/register` returns 404, and `UserRole.PUBLIC` is not assignable. **Public mode** exposes exactly three surfaces and nothing more: (1) a public landing page, (2) a resident-registration path, and (3) an authenticated records-request submission form for `UserRole.PUBLIC` users.

Submission requires authentication – anonymous walk-up submission is intentionally out of scope. Staff roles (ADMIN, STAFF, REVIEWER, READ_ONLY, LIAISON) continue to use /requests/ and receive 403 on the public submit endpoint. An unauthenticated GET /config/portal-mode endpoint (typed PortalModeResponse { mode: Literal["public","private"] }) is always mounted so the frontend can discover the active mode on boot and branch its routing. Fixed a pre-existing bug in UserCreate that forced self-registered users to UserRole.STAFF ; self-registration now correctly forces UserRole.PUBLIC (and is still only reachable in public mode). Coverage: 15 pytest cases in backend/tests/test_portal_mode.py plus 12 vitest cases across PublicLanding.test.tsx , PublicRegister.test.tsx , and PublicSubmit.test.tsx . Explicitly not shipped in this slice and not implied by any copy: published-records search, a full resident dashboard, a track-my-request suite, or any other public-portal feature.

- **T5E – Windows unsigned double-click installer, 2026-04-22 (1d5429d ; test-harness flake fix e898319):** Real Windows .exe installer built with Inno Setup 6.x, produced on every v* tag by .github/workflows/release.yml on windows-latest via choco install innosetup -y + installer/windows/build-installer.sh . **Unsigned by design per Scott-locked B3=α posture** – operators must expect SmartScreen "Windows protected your PC – Unknown publisher" on first run; install path documented in [installer/windows/README.md](#). Flow is split into two Start Menu shortcuts: **Start CivicRecords AI** (daily docker compose up -d) and **Install or Repair CivicRecords AI** (full bootstrap + picker + model pull). Version is tag-derived (no hardcoded version drift) via /DMyAppVersion= from \$CIVICRECORDS_VERSION . Desktop shortcut mirrors daily-start. macOS and Linux remain on the script path (install.sh) – native installer parity on those platforms is explicit follow-on work, not scheduled.
- **T3D regen (bf3c9c3)** – Regenerated docs/openapi.json and frontend/src/generated/api.ts after the T5A schema change; CI's stale-check gate enforces this on every subsequent backend schema or route change.
- **CI hygiene (5dbeed7)** – Bumped actions/checkout@v4 and actions/setup-node@v4 for Node 24 runtime support ahead of the 2026-06-02 GitHub runner default flip.

In v1.2.0 – security hardening carried from earlier sprints (all tagged in this release):

- **T2A** – Role self-escalation via `PATCH /users/me` closed (`UserSelfUpdate` schema); all 24 department-scoped handlers enforce `require_department_scope` ; 404/403 status-code info-leak unified via `require_department_or_404` ; Pattern D list-endpoint fail-open fixed on 4 routes; parameterized enforcement test covers 25 routes
- **T2B** – `connection_config` redacted from `GET /datasources/` for all non-admin users (`DataSourceAdminRead` returned only on admin write endpoints). **ENG-001 runtime exposure: closed.**
- **Tier 6 / ENG-001 – At-rest encryption for `data_sources.connection_config` , 2026-04-23:** Connector credentials are now encrypted at rest using Fernet (AES-128-CBC + HMAC-SHA256) via a transparent SQLAlchemy `EncryptedJSONB` `TypeDecorator`; envelope shape is `{"v": 1, "ct": "<fernet-token>"}` . A new required `ENCRYPTION_KEY` env var (installer auto-generates on fresh installs) drives the encryption; a reversible Alembic migration (`019_encrypt_connection_config`) encrypts existing plaintext rows and decrypts on downgrade. Operator verification: `docker compose run --rm --no-deps api python scripts/verify_at_rest.py` . **ENG-001 now fully closed.** Key rotation is not supported in this release and is tracked as a future slice; the versioned envelope (`"v": 1`) leaves the door open.
- **T2C** – `FIRST_ADMIN_PASSWORD` startup validator (rejects `.env.example` placeholder, empty, <12 chars, and common defaults); SSRF host validator blocks loopback, IMDS, and RFC1918 ranges on REST and ODBC connector URLs at schema-validation time
- **T3A** – Create-user form now POSTs to `/api/admin/users` (was `/api/auth/register` , which silently downgraded submitted roles to STAFF)
- **CI** – 631 backend + 36 frontend tests (CI-verified on PR #61, run 25272823105); bootstrap-failure smoke test confirms stack rejects placeholder admin password; `docs/openapi.json` and `frontend/src/generated/api.ts` stale-check enforced on every build (T3D)

> **ENG-001 standing caveat (historical – closed 2026-04-23):** At-rest encryption for `data_sources.connection_config` landed as Tier 6 / ENG-001. Prior to Tier 6 the `JSONB` column was plaintext and visible to any database superuser, `pg_dump`

output, or restored backup. Post-Tier-6 the column is stored as a Fernet envelope keyed off `ENCRYPTION_KEY`; `pg_dump` output and raw backups contain ciphertext only. T2B runtime exposure closed in an earlier sprint; Tier 6 closes the at-rest gap. ENG-001 is now fully closed. See Section 8.10 of [docs/UNIFIED-SPEC.md](#) and the Operator section on the encryption key in [USER-MANUAL.md](#) for details.

Roadmap (per [canonical spec](#)):

Phase	Focus	Key Deliverables	Status
Phase 0	Design foundation	shadcn/ui, design tokens, sidebar shell, component library	In progress
Phase 1	Staff workbench redesign	Redesign all staff pages with new design system, WCAG 2.2 AA	In progress
Phase 2	New backend features	Fees, notifications (SMTP), response letters, context manager, analytics, liaison role, department scoping, audit retention	Partially built
Phase 3	Public portal	Public homepage, search, guided request wizard, request tracker, help pages	Partial — T5D minimal surface shipped (landing + resident-registration + authenticated submission); published-records search, resident dashboard, and track-my-request remain Planned
Phase 4	Transparency layer	Open records library, reporting dashboards, public archive, federation	Planned (v2.0)

Note: Version numbers (semver) track release history. Phase numbers track design completeness per the canonical spec. They are separate systems. Current build (v1.6.1) includes backend work from Phases 0-2 and partial Phase 3 (T5D minimal public portal surface), but has not completed the full scope of any phase. See [canonical spec](#) for complete requirements and [reconciliation](#) for current gap analysis.